DM&V

Practical Code

Experiments 1 to 6 · Python source, step by step

Index

Exp. No.	Title of Experiment	Page
1	Data Loading, Storage and File Formats	2
2	Interacting with Web APIs	8
3	Data Cleaning and Preparation	16
4	Data Wrangling	26
5	Data Visualization Using Matplotlib	36
6	Data Aggregation	46

EXPERIMENT 1

Data Loading, Storage and File Formats

Analyzing Sales Data from Multiple File Formats (CSV, Excel and JSON)

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Create Sample Sales Data

For this practical, sample sales data is created and saved in three different file formats.

```
data = {
    "Order_ID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    "Product": [
        "Laptop", "Mobile", "Chair", "Table", "Laptop",
        "Mobile", "Headphones", "Chair", "Table", "Laptop"
    ],
    "Category": [
        "Electronics", "Electronics", "Furniture", "Furniture", "Electronics",
        "Electronics", "Electronics", "Furniture", "Furniture", "Electronics"
    ],
    "Quantity": [2, 3, 4, 2, 1, 5, 3, 2, 4, 1],
    "Price": [50000, 20000, 5000, 8000, 50000, 20000, 3000, 5000, 8000, 50000]
}

df = pd.DataFrame(data)

print(df)
```

Step 3: Save Data in CSV, Excel and JSON

```
df.to_csv("sales_data.csv", index=False)
df.to_excel("sales_data.xlsx", index=False)
df.to_json("sales_data.json", orient="records", indent=4)

print("Files created successfully.")
```

Step 4: Load CSV File

```
csv_data = pd.read_csv("sales_data.csv")

print("CSV Data:")
print(csv_data.head())
```

Step 5: Load Excel File

```
excel_data = pd.read_excel("sales_data.xlsx")

print("Excel Data:")
print(excel_data.head())
```

Step 6: Load JSON File

```
json_data = pd.read_json("sales_data.json")

print("JSON Data:")
print(json_data.head())
```

Step 7: Explore the Dataset

```
print("Shape of CSV Data:", csv_data.shape)

print("\nColumn Names:")
print(csv_data.columns)

print("\nData Types:")
print(csv_data.dtypes)

print("\nDataset Information:")
csv_data.info()

print("\nStatistical Summary:")
print(csv_data.describe())
```

Step 8: Check Missing Values

```
print("Missing Values:")
print(csv_data.isnull().sum())
```

Step 9: Check Duplicate Records

```
print("Number of duplicate records:",
      csv_data.duplicated().sum())
```

Step 10: Remove Duplicate Records

```
csv_data = csv_data.drop_duplicates()

print("Duplicates removed.")
print("Number of records:", len(csv_data))
```

Step 11: Handle Missing Values

```
csv_data["Quantity"] = csv_data["Quantity"].fillna(
    csv_data["Quantity"].median()
)

csv_data["Price"] = csv_data["Price"].fillna(
    csv_data["Price"].median()
)

csv_data["Category"] = csv_data["Category"].fillna("Unknown")

print("Missing values handled.")
```

Step 12: Standardize Column Names

```
csv_data.columns = (
    csv_data.columns
    .str.strip()
    .str.replace(" ", "_")
    .str.lower()
)

print(csv_data.columns)
```

Step 13: Convert Data Types

```
csv_data["quantity"] = pd.to_numeric(
    csv_data["quantity"], errors="coerce"
)

csv_data["price"] = pd.to_numeric(
    csv_data["price"], errors="coerce"
)

print(csv_data.dtypes)
```

Step 14: Create Total Sales Column

```
csv_data["total_sales"] = (
    csv_data["quantity"] * csv_data["price"]
)

print(csv_data)
```

Step 15: Unified DataFrame

The CSV, Excel and JSON datasets contain the same structure. Therefore, they can be standardized and combined.

```
csv_data["source"] = "CSV"
excel_data["source"] = "Excel"
json_data["source"] = "JSON"

excel_data.columns = (
    excel_data.columns
    .str.strip()
    .str.replace(" ", "_")
    .str.lower()
)

json_data.columns = (
    json_data.columns
    .str.strip()
    .str.replace(" ", "_")
    .str.lower()
)

excel_data["total_sales"] = (
    excel_data["quantity"] * excel_data["price"]
)

json_data["total_sales"] = (
    json_data["quantity"] * json_data["price"]
)

unified_data = pd.concat(
    [csv_data, excel_data, json_data],
    ignore_index=True
)

print("Unified Data:")
print(unified_data.head())

print("\nShape:", unified_data.shape)
```

Step 16: Remove Duplicate Orders from Unified Data

Since the same sample records were stored in three file formats, concatenating them produces duplicate orders.

```
unified_data = unified_data.drop_duplicates(
    subset=["order_id"]
)

print("Final Unified Dataset:")
print(unified_data)
```

Step 17: Total Sales

```
total_sales = unified_data["total_sales"].sum()

print("Total Sales =", total_sales)
```

Step 18: Average Order Value

```
average_order_value = unified_data["total_sales"].mean()

print("Average Order Value =", average_order_value)
```

Step 19: Category-wise Sales

```
category_sales = unified_data.groupby(
    "category"
)["total_sales"].sum()

print("Category-wise Sales:")
print(category_sales)
```

Step 20: Product-wise Sales

```
product_sales = unified_data.groupby(
    "product"
)["total_sales"].sum().sort_values(ascending=False)

print("Product-wise Sales:")
print(product_sales)
```

Step 21: Descriptive Statistics

```
print(unified_data.describe())
```

Step 22: Bar Plot – Category-wise Sales

```
category_sales.plot(
    kind="bar",
    figsize=(8, 5)
)

plt.title("Category-wise Total Sales")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.xticks(rotation=0)
plt.show()
```

Step 23: Pie Chart – Category Distribution

```
category_counts = unified_data["category"].value_counts()

plt.figure(figsize=(7, 7))

plt.pie(
    category_counts,
    labels=category_counts.index,
```

```
    autopct="%1.1f%%"
)

plt.title("Product Category Distribution")
plt.show()
```

Step 24: Box Plot – Sales Distribution

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    x=unified_data["total_sales"]
)

plt.title("Distribution of Total Sales")
plt.xlabel("Total Sales")
plt.show()
```

Step 25: Final Dataset

```
print("Final Cleaned Sales Dataset:")
print(unified_data)

print("\nTotal Sales:", unified_data["total_sales"].sum())

print(
    "Average Order Value:",
    unified_data["total_sales"].mean()
)

print("\nCategory-wise Sales:")
print(
    unified_data.groupby("category")["total_sales"].sum()
)
```

EXPERIMENT 2

Interacting with Web APIs

Analyzing Weather Data Using OpenWeatherMap API

Step 1: Install Required Libraries

If the libraries are not already installed, run:

```
!pip install requests pandas numpy matplotlib seaborn
```

Step 2: Import Libraries

```
import requests
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 3: Define API Key and Location

Replace YOUR_API_KEY with your actual OpenWeatherMap API key.

```
API_KEY = "YOUR_API_KEY"

CITY = "Pune"
COUNTRY_CODE = "IN"

URL = (
    "https://api.openweathermap.org/data/2.5/weather"
    f"?q={CITY},{COUNTRY_CODE}"
    f"&appid={API_KEY}"
    "&units=metric"
)

print(URL)
```

Note: Do not share your API key publicly or upload it to GitHub.

Step 4: Send Request to OpenWeatherMap API

```
response = requests.get(URL)

print("Status Code:", response.status_code)
```

Step 5: Check API Response

```
if response.status_code == 200:
    weather_data = response.json()
    print("Data retrieved successfully.")
    print(weather_data)
else:
    print("Error:", response.status_code)
    print(response.text)
```

Step 6: Extract Weather Attributes

```
temperature = weather_data["main"]["temp"]
humidity = weather_data["main"]["humidity"]
pressure = weather_data["main"]["pressure"]

wind_speed = weather_data["wind"]["speed"]

weather_description = weather_data["weather"][0]["description"]

print("Temperature:", temperature, "°C")
print("Humidity:", humidity, "%")
print("Pressure:", pressure, "hPa")
print("Wind Speed:", wind_speed, "m/s")
print("Weather:", weather_description)
```

Step 7: Extract Precipitation

Precipitation may not be present in every API response. Therefore, it should be handled safely.

```
rain = weather_data.get("rain", {})
precipitation = rain.get("1h", 0)

print("Precipitation:", precipitation, "mm")
```

If the API does not provide rainfall information, the code assumes 0 mm for the selected observation.

Step 8: Create a DataFrame

```
weather_df = pd.DataFrame({
    "City": [CITY],
    "Temperature": [temperature],
    "Humidity": [humidity],
    "Pressure": [pressure],
    "Wind_Speed": [wind_speed],
    "Precipitation": [precipitation],
    "Weather": [weather_description]
})

print(weather_df)
```

Step 9: Collecting Multiple Weather Observations

```
import time
from datetime import datetime

records = []

for i in range(10):

    response = requests.get(URL)

    if response.status_code == 200:

        data = response.json()

        record = {
            "Timestamp": datetime.now(),
            "City": CITY,
            "Temperature": data["main"].get("temp"),
            "Humidity": data["main"].get("humidity"),
            "Pressure": data["main"].get("pressure"),
            "Wind_Speed": data.get("wind", {}).get("speed"),
            "Precipitation": data.get("rain", {}).get("1h", 0)
        }

        records.append(record)

    else:
        print("API Error:", response.status_code)

    time.sleep(2)

weather_df = pd.DataFrame(records)

print(weather_df)
```

Step 10: Data Cleaning

Check Missing Values

```
print(weather_df.isnull().sum())
```

Remove Duplicate Records

```
weather_df = weather_df.drop_duplicates()
```

Convert Data Types

```
weather_df["Timestamp"] = pd.to_datetime(
    weather_df["Timestamp"]
)

numeric_columns = [
    "Temperature",
```

```
    "Humidity",
    "Pressure",
    "Wind_Speed",
    "Precipitation"
]

for column in numeric_columns:
    weather_df[column] = pd.to_numeric(
        weather_df[column],
        errors="coerce"
    )
```

Handle Missing Values

```
weather_df[numeric_columns] = (
    weather_df[numeric_columns]
    .interpolate()
)

weather_df = weather_df.dropna(
    subset=numeric_columns
)

print(weather_df)
```

Step 11: Convert Timestamp into Date

```
weather_df["Date"] = weather_df["Timestamp"].dt.date

weather_df["Hour"] = weather_df["Timestamp"].dt.hour

print(weather_df)
```

Step 12: Descriptive Statistics

```
print(weather_df[numeric_columns].describe())
```

This provides:

- Count
- Mean
- Standard deviation
- Minimum
- Maximum
- Quartiles

Step 13: Calculate Average Temperature

```
average_temperature = weather_df["Temperature"].mean()

print(
    "Average Temperature:",
    round(average_temperature, 2),
    "°C"
)
```

Step 14: Calculate Maximum and Minimum Temperature

```
max_temperature = weather_df["Temperature"].max()
min_temperature = weather_df["Temperature"].min()

print("Maximum Temperature:", max_temperature, "°C")
print("Minimum Temperature:", min_temperature, "°C")
```

Step 15: Average Humidity and Wind Speed

```
average_humidity = weather_df["Humidity"].mean()
average_wind_speed = weather_df["Wind_Speed"].mean()

print(
    "Average Humidity:",
    round(average_humidity, 2),
    "%"
)

print(
    "Average Wind Speed:",
    round(average_wind_speed, 2),
    "m/s"
)
```

Step 16: Time-Based Aggregation

Daily Weather Statistics

```
daily_weather = weather_df.groupby("Date").agg({
    "Temperature": ["mean", "min", "max"],
    "Humidity": "mean",
    "Wind_Speed": "mean",
    "Precipitation": "sum"
})

print(daily_weather)
```

Step 17: Temperature Trend – Line Chart

```
plt.figure(figsize=(10, 5))

plt.plot(
    weather_df["Timestamp"],
    weather_df["Temperature"],
    marker="o"
)

plt.title("Temperature Changes Over Time")
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Step 18: Humidity Trend

```
plt.figure(figsize=(10, 5))

plt.plot(
    weather_df["Timestamp"],
    weather_df["Humidity"],
    marker="o"
)

plt.title("Humidity Changes Over Time")
plt.xlabel("Time")
plt.ylabel("Humidity (%)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Step 19: Wind Speed Bar Plot

```
plt.figure(figsize=(10, 5))

plt.bar(
    range(len(weather_df)),
    weather_df["Wind_Speed"]
)

plt.title("Wind Speed Variation")
plt.xlabel("Observation")
plt.ylabel("Wind Speed (m/s)")
plt.show()
```

Step 20: Precipitation Plot

```
plt.figure(figsize=(10, 5))

plt.bar(
    range(len(weather_df)),
    weather_df["Precipitation"]
)

plt.title("Precipitation Levels")
plt.xlabel("Observation")
plt.ylabel("Precipitation (mm)")
plt.show()
```

Step 21: Temperature vs Humidity Scatter Plot

```
plt.figure(figsize=(8, 5))

plt.scatter(
    weather_df["Temperature"],
    weather_df["Humidity"]
)

plt.title("Temperature vs Humidity")
plt.xlabel("Temperature (°C)")
plt.ylabel("Humidity (%)")
plt.show()
```

Step 22: Correlation Analysis

```
correlation = weather_df[numeric_columns].corr()

print(correlation)
```

Step 23: Correlation Heatmap

```
plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Weather Attributes Correlation")
plt.show()
```

A correlation value close to:

- +1 indicates strong positive relationship.
- -1 indicates strong negative relationship.

- 0 indicates little or no linear relationship.

Step 24: Save Weather Data

```
weather_df.to_csv(
    "weather_data.csv",
    index=False
)

print("Weather data saved successfully.")
```

EXPERIMENT 3

Data Cleaning and Preparation

Data Cleaning and Preparation for Analyzing Customer Churn in a Telecommunications Company

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

Step 2: Load Dataset

Make sure Telecom_Customer_Churn.csv is present in the same folder as the notebook.

```
df = pd.read_csv("Telecom_Customer_Churn.csv")

print("Dataset loaded successfully.")
print(df.head())
```

Step 3: Explore Dataset

Display Number of Rows and Columns

```
print("Rows and Columns:", df.shape)
```

Display Column Names

```
print("Column Names:")
print(df.columns.tolist())
```

Display First Five Records

```
print(df.head())
```

Display Last Five Records

```
print(df.tail())
```

Dataset Information

```
df.info()
```

Statistical Summary

```
print(df.describe(include="all"))
```

Step 4: Check Missing Values

```
missing_values = df.isnull().sum()

print("Missing Values:")
print(missing_values)
```

To display only columns containing missing values:

```
print(
    df.isnull().sum()[
        df.isnull().sum() > 0
    ]
)
```

Step 5: Handle Missing Values

First, identify numerical and categorical columns.

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns
```

Fill numerical missing values with median:

```
for column in numeric_columns:
    df[column] = df[column].fillna(
        df[column].median()
    )
```

Fill categorical missing values with mode:

```
for column in categorical_columns:
    if df[column].isnull().any():
        df[column] = df[column].fillna(
            df[column].mode()[0]
        )
```

Check again:

```
print("Missing values after cleaning:")
print(df.isnull().sum())
```

Step 6: Remove Duplicate Records

Check the number of duplicate records:

```
print(
    "Number of duplicate records:",
    df.duplicated().sum()
)
```

Remove duplicates:

```
df = df.drop_duplicates()

print(
    "Shape after removing duplicates:",
    df.shape
)
```

Step 7: Standardize Column Names

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

Display the new column names:

```
print(df.columns.tolist())
```

Step 8: Standardize Categorical Values

The following function can be used to remove unwanted spaces and standardize text.

```
for column in df.select_dtypes(
    include="object"
).columns:

    df[column] = (
        df[column]
        .astype(str)
        .str.strip()
    )
```

For example, if a churn column contains different capitalization:

```
if "churn" in df.columns:

    df["churn"] = (
        df["churn"]
        .str.lower()
        .str.strip()
    )
```

```
    df["churn"] = df["churn"].replace({
        "yes": "yes",
        "no": "no",
        "y": "yes",
        "n": "no"
    })
```

Step 9: Convert Columns to Correct Data Types

For example, if the dataset contains total_charges as a text column:

```
if "total_charges" in df.columns:

    df["total_charges"] = pd.to_numeric(
        df["total_charges"],
        errors="coerce"
    )
```

If tenure exists:

```
if "tenure" in df.columns:

    df["tenure"] = pd.to_numeric(
        df["tenure"],
        errors="coerce"
    )
```

Check data types:

```
print(df.dtypes)
```

Step 10: Identify Outliers Using IQR

Select numerical columns:

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns

print(numeric_columns)
```

Calculate the number of outliers:

```
for column in numeric_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    outliers = df[
        (df[column] < lower_bound) |
        (df[column] > upper_bound)
    ]

    print(
        column,
        "-> Outliers:",
        len(outliers)
    )
```

Step 11: Handle Outliers

Instead of deleting potentially useful customers, extreme numerical values can be **capped** at the IQR limits.

```
for column in numeric_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df[column] = df[column].clip(
        lower_bound,
        upper_bound
    )
```

This reduces the effect of extreme values while preserving the observations.

Step 12: Feature Engineering

Feature engineering creates additional variables that may be useful for churn analysis.

12.1 Tenure Group

```
if "tenure" in df.columns:

    df["tenure_group"] = pd.cut(
        df["tenure"],
        bins=[-1, 12, 24, 48, 72],
        labels=[
            "0-1 Year",
            "1-2 Years",
            "2-4 Years",
            "4+ Years"
        ]
    )

print(df[["tenure", "tenure_group"]].head())
```

12.2 Average Monthly Charge

If both total_charges and tenure are available:

```
if "total_charges" in df.columns and "tenure" in df.columns:

    df["avg_monthly_spend"] = (
        df["total_charges"] /
        df["tenure"].replace(0, np.nan)
    )

    df["avg_monthly_spend"] = (
        df["avg_monthly_spend"]
        .replace([np.inf, -np.inf], np.nan)
        .fillna(0)
    )
```

12.3 Number of Services

If service-related columns are available, they can be counted.

```
service_columns = [
    column for column in [
        "phone_service",
        "multiple_lines",
        "online_security",
        "online_backup",
        "device_protection",
        "tech_support",
        "streaming_tv",
        "streaming_movies"
    ]
    if column in df.columns
]

if service_columns:

    df["number_of_services"] = (
        df[service_columns]
        .apply(
            lambda row:
            sum(
                str(value).lower()
                in ["yes", "yes ", "1", "true"]
                for value in row
            ),
            axis=1
        )
    )

print(df.head())
```

Step 13: Encode Churn Variable

For machine learning, the target variable can be converted into binary form.

```
if "churn" in df.columns:

    df["churn"] = (
        df["churn"]
        .astype(str)
        .str.lower()
        .map({
            "yes": 1,
            "no": 0
        })
    )

print(df["churn"].value_counts())
```

Here:

1 = Customer Churned

0 = Customer Did Not Churn

Step 14: Prepare Features and Target

```
X = df.drop(
    columns=["churn"],
    errors="ignore"
)

y = df["churn"]
```

Convert categorical variables into dummy variables:

```
X = pd.get_dummies(
    X,
    drop_first=True
)

print("Feature shape:", X.shape)
```

Step 15: Handle Remaining Missing Values

```
X = X.replace(
    [np.inf, -np.inf],
    np.nan
)

X = X.fillna(
    X.median(numeric_only=True)
)

X = X.fillna(0)
```

Step 16: Train-Test Split

Split the dataset into 80% training data and 20% testing data.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Step 17: Scale Numerical Features

```
scaler = StandardScaler()

numeric_features = X_train.select_dtypes(
    include=np.number
).columns

X_train[numeric_features] = scaler.fit_transform(
    X_train[numeric_features]
)

X_test[numeric_features] = scaler.transform(
    X_test[numeric_features]
)

print("Numerical features scaled successfully.")
```

Step 18: Export Cleaned Dataset

```
df.to_csv(
    "Telecom_Customer_Churn_Cleaned.csv",
    index=False
)

print(
    "Cleaned dataset exported successfully."
)
```

Step 19: Export Training and Testing Data

```
train_data = X_train.copy()
train_data["churn"] = y_train.values

test_data = X_test.copy()
test_data["churn"] = y_test.values
```

```
train_data.to_csv(
    "churn_train.csv",
    index=False
)

test_data.to_csv(
    "churn_test.csv",
    index=False
)

print("Training and testing datasets exported.")
```

Step 20: Visualization of Churn Distribution

```
if "churn" in df.columns:

    sns.countplot(
        x=df["churn"]
    )

    plt.title("Customer Churn Distribution")
    plt.xlabel("Churn")
    plt.ylabel("Number of Customers")
    plt.show()
```

Step 21: Box Plot for Numerical Features

```
if "monthly_charges" in df.columns:

    plt.figure(figsize=(8, 5))

    sns.boxplot(
        x=df["monthly_charges"]
    )

    plt.title("Monthly Charges – Outlier Detection")
    plt.xlabel("Monthly Charges")

    plt.show()
```

Step 22: Correlation Heatmap

```
numeric_df = df.select_dtypes(
    include=np.number
)

plt.figure(figsize=(10, 7))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
```

```
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()
```

EXPERIMENT 4

Data Wrangling

Data Wrangling on Real Estate Market

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import OneHotEncoder
```

Step 2: Import Dataset

```
df = pd.read_csv("RealEstate_Prices.csv")

print("Dataset loaded successfully.")

print(df.head())
```

Step 3: Explore Dataset

Display Shape

```
print("Dataset Shape:", df.shape)
```

Display Column Names

```
print("Original Column Names:")
print(df.columns.tolist())
```

Display Information

```
df.info()
```

Statistical Summary

```
print(df.describe(include="all"))
```

Step 4: Clean Column Names

Column names are converted to lowercase and spaces/special characters are replaced with underscores.

```
df.columns = (
    df.columns
    .str.strip()
```

```
    .str.lower()
    .str.replace(r"[^a-zA-Z0-9]+", "_", regex=True)
    .str.strip("_")
)

print("Cleaned Column Names:")
print(df.columns.tolist())
```

For example:

```
Sale Price → sale_price
Property Type → property_type
Lot Area (sq.ft.) → lot_area_sq_ft
```

Step 5: Identify Missing Values

```
print("Missing Values:")
print(df.isnull().sum())
```

Display only columns with missing values:

```
missing = df.isnull().sum()

print(
    missing[missing > 0]
)
```

Step 6: Handle Missing Values

Identify numerical and categorical columns:

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns
```

Fill numerical missing values with median:

```
for column in numeric_columns:

    df[column] = df[column].fillna(
        df[column].median()
    )
```

Fill categorical missing values with mode:

```
for column in categorical_columns:

    if df[column].isnull().any():
```

```
        df[column] = df[column].fillna(
            df[column].mode()[0]
        )
```

Verify:

```
print("Missing values after cleaning:")
print(df.isnull().sum())
```

Step 7: Remove Duplicate Records

Check duplicate records:

```
print(
    "Number of duplicates:",
    df.duplicated().sum()
)
```

Remove duplicates:

```
df = df.drop_duplicates()

print(
    "Shape after removing duplicates:",
    df.shape
)
```

Step 8: Standardize Text Data

Remove unnecessary spaces from categorical columns.

```
for column in df.select_dtypes(
    include="object"
).columns:

    df[column] = (
        df[column]
        .astype(str)
        .str.strip()
    )
```

Step 9: Convert Numeric Columns

If columns such as sale_price or area are stored as strings, convert them into numeric values.

```
possible_numeric = [
    "sale_price",
    "price",
    "area",
    "square_feet",
    "lot_area",
    "bedrooms",
    "bathrooms"
```

```
]

for column in possible_numeric:

    if column in df.columns:

        df[column] = pd.to_numeric(
            df[column],
            errors="coerce"
        )
```

Check data types:

```
print(df.dtypes)
```

Step 10: Data Merging

If an additional dataset such as neighborhood information is available, it can be merged with the real estate dataset.

Example additional dataset:

```
neighborhood_data = pd.DataFrame({
    "neighborhood": [
        "A",
        "B",
        "C"
    ],
    "population": [
        10000,
        15000,
        12000
    ],
    "average_income": [
        60000,
        75000,
        65000
    ]
})

print(neighborhood_data)
```

Merge using a common column:

```
if "neighborhood" in df.columns:

    merged_df = pd.merge(
        df,
        neighborhood_data,
        on="neighborhood",
        how="left"
    )

else:

    merged_df = df.copy()
```

```
print(merged_df.head())
```

The left merge ensures that all real estate records are retained.

Step 11: Filtering and Subsetting

11.1 Filter by Property Type

```
if "property_type" in df.columns:

    house_data = df[
        df["property_type"]
        .astype(str)
        .str.lower()
        .str.contains("house")
    ]

    print(house_data.head())
```

11.2 Filter by Location

```
if "neighborhood" in df.columns:

    selected_area = df[
        df["neighborhood"]
        .astype(str)
        .str.lower()
        == "a"
    ]

    print(selected_area)
```

11.3 Filter by Sale Price

```
if "sale_price" in df.columns:

    expensive_properties = df[
        df["sale_price"] > df["sale_price"].median()
    ]

    print(expensive_properties.head())
```

Step 12: Categorical Variables

Display categorical columns:

```
categorical_columns = df.select_dtypes(
    include="object"
).columns

print("Categorical Columns:")
print(categorical_columns.tolist())
```

Step 13: One-Hot Encoding

One-hot encoding converts categorical variables into binary numerical columns.

```
encoded_df = pd.get_dummies(
    df,
    columns=categorical_columns,
    drop_first=True
)

print(encoded_df.head())
```

Step 14: Aggregation

14.1 Average Sale Price by Neighborhood

```
if "neighborhood" in df.columns and "sale_price" in df.columns:

    neighborhood_summary = (
        df.groupby("neighborhood")
        ["sale_price"]
        .agg(
            ["count", "mean", "median", "min", "max"]
        )
        .sort_values(
            "mean",
            ascending=False
        )
    )

    print(neighborhood_summary)
```

Step 15: Average Sale Price by Property Type

```
if "property_type" in df.columns and "sale_price" in df.columns:

    property_summary = (
        df.groupby("property_type")
        ["sale_price"]
        .agg(
            ["count", "mean", "median", "min", "max"]
        )
        .sort_values(
            "mean",
            ascending=False
        )
    )

    print(property_summary)
```

Step 16: Identify Outliers Using IQR

Select numerical columns:

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns

print(numeric_columns.tolist())
```

Calculate outliers:

```
for column in numeric_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[column] < lower_bound) |
        (df[column] > upper_bound)
    ]

    print(
        column,
        "-> Outliers:",
        len(outliers)
    )
```

Step 17: Handle Outliers

Extreme values can be capped using the IQR boundaries.

```
for column in numeric_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df[column] = df[column].clip(
        lower_bound,
        upper_bound
    )
```

Step 18: Calculate Price per Square Foot

If both sale price and area are available, a useful derived feature can be created.

```
if "sale_price" in df.columns and "area" in df.columns:

    df["price_per_sq_ft"] = (
        df["sale_price"] /
        df["area"].replace(0, np.nan)
    )

    df["price_per_sq_ft"] = (
        df["price_per_sq_ft"]
        .replace([np.inf, -np.inf], np.nan)
        .fillna(0)
    )

print(df.head())
```

Step 19: Visualization – Average Price by Property Type

```
if "property_type" in df.columns and "sale_price" in df.columns:

    plt.figure(figsize=(10, 5))

    (
        df.groupby("property_type")["sale_price"]
        .mean()
        .sort_values()
        .plot(kind="bar")
    )

    plt.title("Average Sale Price by Property Type")
    plt.xlabel("Property Type")
    plt.ylabel("Average Sale Price")
    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()
```

Step 20: Visualization – Price Distribution

```
if "sale_price" in df.columns:

    plt.figure(figsize=(8, 5))

    sns.histplot(
        df["sale_price"],
        kde=True
    )

    plt.title("Distribution of Real Estate Sale Prices")
    plt.xlabel("Sale Price")
```

```
    plt.ylabel("Frequency")

    plt.show()
```

Step 21: Visualization – Area vs Sale Price

```
if "area" in df.columns and "sale_price" in df.columns:

    plt.figure(figsize=(8, 5))

    sns.scatterplot(
        data=df,
        x="area",
        y="sale_price"
    )

    plt.title("Area vs Sale Price")
    plt.xlabel("Area")
    plt.ylabel("Sale Price")

    plt.show()
```

Step 22: Correlation Analysis

```
numeric_df = df.select_dtypes(
    include=np.number
)

print(
    numeric_df.corr()
)
```

Heatmap:

```
plt.figure(figsize=(10, 7))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Real Estate Features")

plt.show()
```

Step 23: Export Cleaned Dataset

```
df.to_csv(
    "RealEstate_Prices_Cleaned.csv",
    index=False
)

print(
    "Cleaned dataset exported successfully."
)
```

EXPERIMENT 5

Data Visualization Using Matplotlib

Analyzing Air Quality Index (AQI) Trends in a City Using Matplotlib

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Import Dataset

```
df = pd.read_csv("City_Air_Quality.csv")

print("Dataset loaded successfully.")
print(df.head())
```

Step 3: Explore Dataset

Display Shape

```
print("Dataset Shape:", df.shape)
```

Display Columns

```
print("Column Names:")
print(df.columns.tolist())
```

Display Dataset Information

```
df.info()
```

Statistical Summary

```
print(df.describe(include="all"))
```

Step 4: Clean Column Names

To make the column names easier to use:

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(r"[^a-zA-Z0-9]+", "_", regex=True)
    .str.strip("_")
)
```

```
print(df.columns.tolist())
```

For example:

```
Date Time → date_time
PM2.5 → pm2_5
PM10 → pm10
AQI Value → aqi_value
```

Step 5: Convert Date/Time Column

The dataset may contain either a combined date-time column or separate date and time columns.

If date_time exists:

```
if "date_time" in df.columns:

    df["date_time"] = pd.to_datetime(
        df["date_time"],
        errors="coerce"
    )
```

If separate date and time columns exist:

```
elif "date" in df.columns and "time" in df.columns:

    df["date_time"] = pd.to_datetime(
        df["date"].astype(str) + " " +
        df["time"].astype(str),
        errors="coerce"
    )
```

Step 6: Check Missing Values

```
print("Missing Values:")
print(df.isnull().sum())
```

For numerical columns:

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns

df[numeric_columns] = (
    df[numeric_columns]
    .interpolate()
)
```

Step 7: Convert Pollutant Columns to Numeric

```
pollutant_columns = [
    "pm2_5",
    "pm10",
    "co"
]

for column in pollutant_columns:

    if column in df.columns:

        df[column] = pd.to_numeric(
            df[column],
            errors="coerce"
        )
```

Convert AQI:

```
if "aqi" in df.columns:

    df["aqi"] = pd.to_numeric(
        df["aqi"],
        errors="coerce"
    )

elif "aqi_value" in df.columns:

    df["aqi_value"] = pd.to_numeric(
        df["aqi_value"],
        errors="coerce"
    )
```

Step 8: Identify AQI Column

```
if "aqi" in df.columns:

    aqi_column = "aqi"

elif "aqi_value" in df.columns:

    aqi_column = "aqi_value"

else:

    raise ValueError(
        "AQI column not found in dataset."
    )

print("AQI Column:", aqi_column)
```

Step 9: Overall AQI Trend – Line Plot

```
plt.figure(figsize=(12, 5))

plt.plot(
    df["date_time"],
    df[aqi_column],
    marker="o",
    linewidth=1.5,
    label="AQI"
)

plt.title("Overall AQI Trend Over Time")
plt.xlabel("Date and Time")
plt.ylabel("AQI")
plt.legend()
plt.grid(True)

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```

Step 10: PM2.5 Trend

```
if "pm2_5" in df.columns:

    plt.figure(figsize=(12, 5))

    plt.plot(
        df["date_time"],
        df["pm2_5"],
        marker="o",
        label="PM2.5"
    )

    plt.title("PM2.5 Trend Over Time")
    plt.xlabel("Date and Time")
    plt.ylabel("PM2.5")
    plt.legend()
    plt.grid(True)

    plt.xticks(rotation=45)
    plt.tight_layout()

    plt.show()
```

Step 11: PM10 Trend

```
if "pm10" in df.columns:

    plt.figure(figsize=(12, 5))
```

```
    plt.plot(
        df["date_time"],
        df["pm10"],
        marker="o",
        label="PM10"
    )

    plt.title("PM10 Trend Over Time")
    plt.xlabel("Date and Time")
    plt.ylabel("PM10")
    plt.legend()
    plt.grid(True)

    plt.xticks(rotation=45)
    plt.tight_layout()

    plt.show()
```

Step 12: CO Trend

```
if "co" in df.columns:

    plt.figure(figsize=(12, 5))

    plt.plot(
        df["date_time"],
        df["co"],
        marker="o",
        label="CO"
    )

    plt.title("CO Trend Over Time")
    plt.xlabel("Date and Time")
    plt.ylabel("CO")
    plt.legend()
    plt.grid(True)

    plt.xticks(rotation=45)
    plt.tight_layout()

    plt.show()
```

Step 13: Combined Pollutant Trends

```
plt.figure(figsize=(12, 6))

for column in pollutant_columns:

    if column in df.columns:

        plt.plot(
            df["date_time"],
```

```
            df[column],
            marker="o",
            label=column.upper()
        )

plt.title("Pollutant Levels Over Time")
plt.xlabel("Date and Time")
plt.ylabel("Pollutant Level")
plt.legend()
plt.grid(True)

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```

Step 14: Bar Plot – AQI by Date

First calculate daily average AQI:

```
df["date"] = df["date_time"].dt.date

daily_aqi = (
    df.groupby("date")[aqi_column]
    .mean()
)

print(daily_aqi)
```

Create bar plot:

```
plt.figure(figsize=(12, 6))

plt.bar(
    range(len(daily_aqi)),
    daily_aqi.values
)

plt.title("Average Daily AQI")
plt.xlabel("Date")
plt.ylabel("Average AQI")

plt.xticks(
    range(len(daily_aqi)),
    daily_aqi.index,
    rotation=45
)

plt.tight_layout()
plt.show()
```

Step 15: Box Plot – AQI Distribution

```
plt.figure(figsize=(8, 5))

plt.boxplot(
    df[aqi_column].dropna()
)

plt.title("AQI Distribution")
plt.ylabel("AQI")

plt.grid(True)
plt.show()
```

Step 16: Box Plot for Pollutants

```
available_pollutants = [
    column for column in pollutant_columns
    if column in df.columns
]

data_to_plot = [
    df[column].dropna()
    for column in available_pollutants
]

plt.figure(figsize=(10, 6))

plt.boxplot(
    data_to_plot,
    labels=[
        column.upper()
        for column in available_pollutants
    ]
)

plt.title("Distribution of Pollutant Levels")
plt.xlabel("Pollutant")
plt.ylabel("Concentration")

plt.grid(True)
plt.show()
```

Step 17: Scatter Plot – AQI vs PM2.5

```
if "pm2_5" in df.columns:

    plt.figure(figsize=(8, 5))

    plt.scatter(
        df["pm2_5"],
        df[aqi_column],
        alpha=0.7
```

```
    )

    plt.title("AQI vs PM2.5")
    plt.xlabel("PM2.5")
    plt.ylabel("AQI")

    plt.grid(True)
    plt.show()
```

Step 18: Scatter Plot – AQI vs PM10

```
if "pm10" in df.columns:

    plt.figure(figsize=(8, 5))

    plt.scatter(
        df["pm10"],
        df[aqi_column],
        alpha=0.7
    )

    plt.title("AQI vs PM10")
    plt.xlabel("PM10")
    plt.ylabel("AQI")

    plt.grid(True)
    plt.show()
```

Step 19: Scatter Plot – AQI vs CO

```
if "co" in df.columns:

    plt.figure(figsize=(8, 5))

    plt.scatter(
        df["co"],
        df[aqi_column],
        alpha=0.7
    )

    plt.title("AQI vs CO")
    plt.xlabel("CO")
    plt.ylabel("AQI")

    plt.grid(True)
    plt.show()
```

Step 20: Correlation Analysis

```
correlation_columns = [
    aqi_column
] + [
    column for column in pollutant_columns
    if column in df.columns
]

correlation = df[
    correlation_columns
].corr()

print("Correlation Matrix:")
print(correlation)
```

Step 21: Correlation Visualization

Although the main practical uses Matplotlib, the correlation matrix can also be displayed using Matplotlib's imshow().

```
plt.figure(figsize=(8, 6))

plt.imshow(
    correlation,
    interpolation="nearest"
)

plt.colorbar()

plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=45
)

plt.yticks(
    range(len(correlation.columns)),
    correlation.columns
)

plt.title("Correlation Between AQI and Pollutants")

plt.tight_layout()
plt.show()
```

Step 22: AQI Summary Statistics

```
print("Average AQI:",
      df[aqi_column].mean())

print("Maximum AQI:",
      df[aqi_column].max())
```

```
print("Minimum AQI:",
      df[aqi_column].min())

print("Median AQI:",
      df[aqi_column].median())
```

EXPERIMENT 6

Data Aggregation

Analyzing Sales Performance by Region in a Retail Company Using Data Aggregation

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Import Dataset

```
df = pd.read_csv("Retail_Sales_Data.csv")

print("Dataset loaded successfully.")

print(df.head())
```

Step 3: Explore Dataset

Display Shape

```
print("Dataset Shape:", df.shape)
```

Display Column Names

```
print("Column Names:")
print(df.columns.tolist())
```

Display Information

```
df.info()
```

Statistical Summary

```
print(df.describe(include="all"))
```

Step 4: Clean Column Names

To make column names easier to use:

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(r"[^a-zA-Z0-9]+", "_", regex=True)
    .str.strip("_")
)
```

```
print("Cleaned Column Names:")
print(df.columns.tolist())
```

For example:

```
Region          → region
Product Category → product_category
Sales Amount    → sales_amount
Transaction Date → transaction_date
```

Step 5: Check Missing Values

```
print("Missing Values:")
print(df.isnull().sum())
```

Remove rows containing missing values in important columns:

```
required_columns = [
    "region",
    "product_category",
    "sales_amount"
]

df = df.dropna(
    subset=required_columns
)

print("Dataset Shape After Cleaning:", df.shape)
```

Step 6: Remove Duplicate Records

```
print(
    "Duplicate Records:",
    df.duplicated().sum()
)
```

Remove duplicates:

```
df = df.drop_duplicates()

print(
    "Dataset Shape After Removing Duplicates:",
    df.shape
)
```

Step 7: Convert Sales Amount to Numeric

```
df["sales_amount"] = pd.to_numeric(
    df["sales_amount"],
    errors="coerce"
)
```

Remove invalid sales records:

```
df = df.dropna(
    subset=["sales_amount"]
)

print(df.head())
```

Step 8: Explore Relevant Variables

```
print("Regions:")
print(df["region"].unique())

print("\nProduct Categories:")
print(df["product_category"].unique())

print("\nTotal Transactions:",
      len(df))
```

Step 9: Total Sales by Region

Group the data by region and calculate total sales.

```
regional_sales = (
    df.groupby("region")["sales_amount"]
    .sum()
    .sort_values(ascending=False)
)

print("Total Sales by Region:")
print(regional_sales)
```

Example output:

```
North       1250000
West        1100000
South        950000
East         850000
```

The actual values depend on the supplied dataset.

Step 10: Calculate Additional Regional Statistics

We can calculate multiple statistics at the same time.

```
regional_summary = (
    df.groupby("region")
    ["sales_amount"]
```

```
    .agg(
        total_sales="sum",
        average_sales="mean",
        minimum_sales="min",
        maximum_sales="max",
        transaction_count="count"
    )
)

print(regional_summary)
```

Step 11: Bar Plot – Sales by Region

```
plt.figure(figsize=(10, 6))

regional_sales.plot(
    kind="bar"
)

plt.title("Total Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales Amount")

plt.xticks(rotation=0)
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

Step 12: Pie Chart – Regional Sales Distribution

```
plt.figure(figsize=(8, 8))

plt.pie(
    regional_sales.values,
    labels=regional_sales.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Sales Distribution by Region")

plt.show()
```

Step 13: Identify Top-Performing Region

```
top_region = regional_sales.idxmax()
top_sales = regional_sales.max()

print("Top Performing Region:", top_region)
print("Total Sales:", top_sales)
```

Step 14: Find Top 3 Regions

```
top_3_regions = regional_sales.head(3)

print("Top 3 Performing Regions:")
print(top_3_regions)
```

Step 15: Sales by Region and Product Category

Group the data by both region and product category.

```
region_category_sales = (
    df.groupby(
        ["region", "product_category"]
    )["sales_amount"]
    .sum()
)

print(
    "Sales by Region and Product Category:"
)

print(region_category_sales)
```

Step 16: Convert to Pivot Table

A pivot table makes the aggregated results easier to visualize.

```
sales_pivot = (
    df.pivot_table(
        index="region",
        columns="product_category",
        values="sales_amount",
        aggfunc="sum",
        fill_value=0
    )
)

print(sales_pivot)
```

Example structure:

```
product_category    Clothing    Electronics    Grocery
region
East                  ...          ...          ...
North                 ...          ...          ...
South                 ...          ...          ...
West                  ...          ...          ...
```

Step 17: Stacked Bar Plot

```
sales_pivot.plot(
    kind="bar",
    stacked=True,
    figsize=(12, 7)
)

plt.title(
    "Sales by Region and Product Category"
)

plt.xlabel("Region")
plt.ylabel("Total Sales Amount")

plt.xticks(rotation=0)
plt.legend(
    title="Product Category"
)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```

Step 18: Grouped Bar Plot

A grouped bar chart can also be used to compare categories.

```
sales_pivot.plot(
    kind="bar",
    figsize=(12, 7)
)

plt.title(
    "Comparison of Sales by Region and Product Category"
)

plt.xlabel("Region")
plt.ylabel("Total Sales Amount")

plt.xticks(rotation=0)
plt.legend(
    title="Product Category"
)

plt.grid(
    axis="y",
    alpha=0.3
)
```

```
plt.tight_layout()
plt.show()
```

Step 19: Total Sales by Product Category

```
category_sales = (
    df.groupby("product_category")
    ["sales_amount"]
    .sum()
    .sort_values(ascending=False)
)

print(
    "Total Sales by Product Category:"
)

print(category_sales)
```

Visualization:

```
plt.figure(figsize=(10, 6))

category_sales.plot(
    kind="bar"
)

plt.title(
    "Total Sales by Product Category"
)

plt.xlabel("Product Category")
plt.ylabel("Total Sales Amount")

plt.xticks(rotation=45)
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

Step 20: Percentage Contribution of Each Region

```
regional_percentage = (
    regional_sales /
    regional_sales.sum()
) * 100

print(
    "Regional Sales Contribution (%):"
)

print(regional_percentage.round(2))
```

Step 21: Overall Sales Statistics

```
total_sales = df["sales_amount"].sum()
average_sales = df["sales_amount"].mean()
maximum_sale = df["sales_amount"].max()
minimum_sale = df["sales_amount"].min()

print("Total Sales:", total_sales)
print("Average Transaction Sales:", average_sales)
print("Maximum Transaction:", maximum_sale)
print("Minimum Transaction:", minimum_sale)
```